



Give Your API a Real Database

Week 10 Laboratory · ORM Integration & Secrets Management

SQLAlchemy

.env secrets

FastAPI

GOAL

Retire the in-memory list

Your Projects API works, but its data lives in a Python list and vanishes on every restart. Today you connect it to a real **PostgreSQL** database through the **SQLAlchemy ORM**, and you move the database password out of the code into a **.env** file. When you finish, your data survives a restart and no secret lives in your source.

You are given a starter project

The `orm-lab/backend` folder is a working FastAPI app that still uses a list. It has six numbered `TODO`s across three files. Do them in order.

- `database.py` — connection + secret (TODO 1, 3)
- `models.py` — the table as a class (TODO 2)
- `main.py` — the endpoints (TODO 4, 5, 6)

Set up

terminal

```
cd backend
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env              # then edit .env with your real values
fastapi dev main.py               # http://localhost:8000/docs
```

Before you write any code — the secret

Open `.env.example`, copy it to `.env`, and put your real database URL there. Confirm `.env` is listed in `.gitignore`. The password must never appear in a `.py` file or in Git.

Time

One lab session

Work

Individually (pairs may discuss)

Needs

Python 3.10+, PostgreSQL running locally

Six steps to a real database

TODO 1 Read the secret from the environment

In `database.py`, load the `.env` file and read the URL from the environment. Replace the `None` placeholder.

```
database.py

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL") # not the real URL, pasted
```

TODO 2 Describe the table as a model

In `models.py`, finish the `Project` class so it maps to a `projects` table with the four columns.

```
models.py

class Project(Base):
    __tablename__ = "projects"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str]
    tech: Mapped[str]
    done: Mapped[bool] = mapped_column(default=False)
```

TODO 3 Create the engine and session

Still in `database.py`, create the engine from the URL, a `Session` factory, and let `init_db()` create the tables.

```
database.py

engine = create_engine(DATABASE_URL)
Session = sessionmaker(bind=engine)

def init_db():
    Base.metadata.create_all(engine)
```

TODO 4 Read from the database

In `main.py`, import `Session` and `Project`, call `init_db()` once, and make `GET /projects` read through a session.

```
main.py

@app.get("/projects")
def list_projects():
    with Session() as session:
        return session.query(Project).all()
```

TODO 5 Create through the ORM

Make `POST /projects` build a `Project`, add it, and commit. Remember: nothing is saved until you `commit()`.

```
main.py

@app.post("/projects", status_code=201)
def create_project(data: ProjectIn):
    with Session() as session:
        new = Project(title=data.title, tech=data.tech)
        session.add(new)
        session.commit()
        session.refresh(new)
        return new
```

TODO 6 Update, then clean up

Make `PATCH /projects/{id}` load the project (404 if missing), flip `done`, and commit. Then **delete the old `projects = [...]` list** — nothing should reference it anymore.

```
main.py

@app.patch("/projects/{project_id}")
def toggle_done(project_id: int):
    with Session() as session:
        p = session.get(Project, project_id)
        if p is None:
            raise HTTPException(404, "Project not found")
        p.done = not p.done
        session.commit()
        session.refresh(p)
        return p
```

FINISH

Done when...

- The app starts with no errors and `/docs` lists your routes.
- You can **create** a project, **restart the server**, and it is still there.
- **Toggling** `done` works, and a missing id returns a real **404**.
- The `projects = [...]` list is gone; nothing references it.
- There is **no password in any .py file**, and `git status` never shows `.env`.

Prove it persists

The whole point of this lab is persistence. Add a project through `/docs`, stop the server with Ctrl+C, start it again, and call `GET /projects`. Your project is still there because it now lives in PostgreSQL, not in memory.

Stretch goals

- Add `GET /projects/{id}` (404 if missing) and `DELETE /projects/{id}`.
- Add a second secret to `.env` (say `APP_NAME`) and read it with `os.getenv`.
- Deliberately hard-code a fake password in a branch, commit it, then practise the safe response: remove it *and* rotate it. Explain in one line why deleting the line is not enough.

FOR MARKING

Rubric

Criterion	Points
Secret read from <code>.env</code> via <code>os.getenv</code> ; nothing hard-coded; <code>.env</code> git-ignored	25
<code>Project</code> model correct (table name, four columns, primary key)	15
Engine + Session created; tables created on startup	15
<code>GET</code> reads through a session	10
<code>POST</code> creates and commits	15
<code>PATCH</code> loads-or-404, toggles, commits	10
In-memory list removed; data proven to survive a restart	10
Total	100